Unidade 4 — UML (Aulas 27–37)

Material didático da disciplina **Engenharia de Software** (CST em ADS · IFRO · 2026/2), cobrindo as **aulas 27 a 37** do Plano de Curso.

> É a unidade que mais alimenta o **Milestone 2** (20/10), onde os diagramas UML valem **20 dos 100 pontos** — e seguem valendo 20 no Milestone 3.

Aulas	Tema	Tipo
27	UML I — visão geral; estruturais × comportamentais	Teórica
28	UML II — casos de uso (a partir das Histórias de Usuário)	Teórica
29–30	UML III e IV — classes: elementos, relacionamentos, cardinalidade, herança	Teórica
31	Laboratório UML I — modelagem do domínio do projeto	Prática
32	UML V — diagrama de sequência	Teórica
33	UML VI — diagrama de atividades	Teórica
34	UML VII — diagrama de estados	Teórica
35–36	Laboratório UML II e revisão pré-monitoria do Milestone 2	Prática
37	Laboratório UML III — revisão cruzada UML × protótipo × requisitos	Prática

> Nível cognitivo (PPC): *Conhecer*, *Compreender*, ***Aplicar***, ***Analisar*** e ***Avaliar*** — é a unidade de nível cognitivo mais alto da disciplina.

Conteúdo da pasta

Arquivo	O que é	Como usar
`01-material-de-leitura.md`	Texto completo com diagramas e mapa de fontes	Leitura antes/depois das aulas
`02-slides.md`	60 slides para projeção, 24 do núcleo + 36 do caso Vez (← → navega, `F` tela cheia)	Exposição em sala
`03-atividade-ES-04.md`	Atividade avaliativa (100 pts)	Aplicação individual
`04-gabarito-ES-04.md`	**Gabarito: 9 defeitos plantados, rubricas e erros típicos**	⚠️ Uso do professor
`AVA-descricao-tarefa-ES-04.md`	Texto pronto para colar na tarefa do AVA	Publicação
`05-laboratorio-vez.md`	**Laboratório sem nota**: 10 exercícios com diagramas do Vez (achar defeito, completar, ler, Gherkin)	Aula de laboratório, em equipe
`06-gabarito-laboratorio-vez.md`	**Gabarito do laboratório**: respostas, defeitos por figura, erros típicos	⚠️ Uso do professor
`../assets/leitor.css` · `../assets/leitor.js`	Estilo e leitor compartilhados (substituem o antigo `assets/doc.css` desta pasta)	—
`diagramas/`	Fontes dos diagramas (SVG)	Ver abaixo

> **HTML anterior:** a versão em HTML desta pasta foi arquivada em `../../aulas-html-legado/unidade-04-uml/` (fora do deploy). A fonte de verdade agora é o `.md`; abra pelo `index.html` de `aulas/`.

> **Os gabaritos não devem ser publicados no AVA nem no deploy** antes do encerramento do prazo. O `deploy.sh` os exclui por padrão (`--com-gabarito` publica).

Diagramas (`diagramas/`)

Arquivo	Conteúdo	Onde é usado
`panorama-uml.svg`	Os 14 diagramas divididos em estruturais × comportamentais, com os 5 da disciplina destacados	material · slides
`casos-de-uso.svg`	Atores, fronteira, casos de uso, «include» e «extend»	material · slides
`classes.svg`	Anatomia da classe, visibilidade, multiplicidade e os 5 relacionamentos	material · slides
`sequencia.svg`	Linhas de vida, ativações, mensagens, retornos e fragmento `alt`	material · slides
`atividades-estados.svg`	Atividades (decisão, fork/join) × máquina de estados, lado a lado	material · slides
`rastreabilidade-uml.svg`	De onde sai cada diagrama e a verificação cruzada da aula 37	material · slides
`atividade-classes.svg`	**Diagrama de classes com 9 defeitos plantados**	atividade (Parte A)

O caso Vez (`vez-*.svg` , 33 arquivos)

O **Vez** (sistema de filas e senhas do próprio professor, em produção) atravessa a unidade como exemplo real. Os diagramas foram **redesenhados** a partir dos do projeto (simplificados, com notação UML conferida), sem copiar dados sensíveis. O que o Vez não tem (ex.: herança no domínio, `loop` na renovação de sessão) aparece **rotulado como hipótese didática**.

Prefixo	Diagramas	Pergunta que responde
`vez-caso-`	O Vez numa tela	O que é e quem usa?
`vez-uc-`	visão geral, «include»/«extend», generalização de atores	Quem faz o quê?
`vez-dominio-`	recorte do domínio, agregação × composição × herança, domínio × banco	Como as peças se relacionam?
`vez-camadas-`	herança de repositórios, dependência entre camadas, middlewares e erro	Como o código se organiza?
`vez-seq-auth-`	login, `alt`, renovação de sessão (401), Google, logout	Em que ordem conversam? (autenticação)
`vez-seq-senha-`	retirar, chamar, multietapas, projeção na TV	Em que ordem conversam? (negócio)
`vez-atividade-`	criação de senha, fork/join, raias	Qual o fluxo de decisões?
`vez-estados-`	estados da Senha, estados de tela, errado × certo	Em que estados vive?
`vez-malha-`	rastreabilidade, mapa de perguntas, análise de impacto	O modelo está consistente?
`vez-lab-`	5 figuras **com defeitos plantados**	laboratório

IDs usados nos exemplos (didáticos, o Vez não numera): requisitos `V-RF-01..05` e `V-RNF-01..02`; histórias `US-V01..V05`; casos de uso com a numeração do diagrama do Vez (`UC06`, `UC12` …); sequência `V-SQ-nn` e testes `V-CT-nn`.

Três diagramas adicionais estão **embutidos nos slides** (US → caso de uso, leitura de multiplicidade nos dois sentidos, e Gherkin → sequência), por serem específicos da projeção.

Atividade ES-04

Parte	Pts	Natureza	Contexto
A — revisar diagrama de classes	30	gabarito fechado	caso fictício
B — casos de uso a partir de US	20	gabarito fechado	caso fictício
C — sequência e máquina de estados	20	rubrica com referência	caso fictício
D — modelar o próprio projeto	30	rubrica	**projeto do grupo**

50 pontos de correção fechada + 50 de rubrica. Diagramas à mão são aceitos — avalia-se o raciocínio de modelagem, não o domínio da ferramenta.

As duas armadilhas da Parte B

- **US-005** ("receber aviso quando a reserva for confirmada") **não deveria virar caso de uso**: o ator é passivo, não inicia a interação. Vale 6 pontos e separa quem entendeu o conceito.
- O diagrama de classes da Parte A tem **quatro defeitos de alta gravidade** ligados ao mesmo eixo — *modelar o domínio × modelar o banco*. Aluno que só aponta defeitos de nomenclatura fica limitado a 14 dos 24 pontos.

Rastreabilidade de fontes

Tópico	Fonte
Os três modos de usar UML (esboço, planta, linguagem)	FOWLER · **Cap. 1** (p. 25) e **Cap. 2** (p. 39)
Panorama e classificação dos diagramas	FOWLER · Cap. 1 · VALENTE · §4.2.2 · SOMMERVILLE **10ed** · Cap. 5
Por que modelar	VALENTE · §4.1 · SOMMERVILLE 10ed · Cap. 5
Casos de uso, «include», «extend»	FOWLER · **Cap. 9** (p. 104) · SOMMERVILLE 10ed · §5.2.1 · PRESSMAN **9ed** · §7.4
Classes — elementos básicos	FOWLER · **Cap. 3** (p. 52) · SOMMERVILLE 10ed · §5.3.1
Classes — conceitos avançados	FOWLER · **Cap. 5** (p. 77)
Generalização e agregação	SOMMERVILLE 10ed · §5.3.2 e §5.3.3 · VALENTE · §4.3.1 a §4.3.3
Diagrama de sequência	FOWLER · **Cap. 4** (p. 67) · SOMMERVILLE 10ed · §5.2.2 · VALENTE · §4.5
Diagrama de atividades	FOWLER · **Cap. 11** (p. 118) · VALENTE · §4.6
Máquina de estados	FOWLER · **Cap. 10** (p. 110) · SOMMERVILLE 10ed · §5.4
Modelos comportamentais	SOMMERVILLE 10ed · §5.4 (§5.4.1, §5.4.2)
Notação normativa	OMG · *UML* 2.5.1 · omg.org/spec/UML — em inglês

A unidade mais bem servida do semestre. O Fowler cobre os cinco diagramas com capítulo e página conferidos no PDF; o Sommerville dá a visão de modelagem de sistemas; o Valente oferece leitura alternativa gratuita e em português. A especificação da OMG entra como referência de consulta, não como leitura de estudo.

Bibliografia base

- FOWLER, Martin. *UML Essencial*, 3. ed. — cap. 1 a 5, 9, 10 e 11.
- SOMMERVILLE, Ian. *Engenharia de Software*, 10. ed. — cap. 5.
- VALENTE, Marco Túlio. *Engenharia de Software Moderna* — **cap. 4 Modelos** · engsoftmoderna.info
- PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de Software*, 9. ed. — §7.4.
- BOOCH; RUMBAUGH; JACOBSON. *UML: guia do usuário.* Elsevier, 2012 (PPC).
- GUEDES, Gilleanes T. A. *UML 2: uma abordagem prática.* Novatec, 2011 (PPC).
- OMG. *Unified Modeling Language (UML), Version 2.5.1* — omg.org/spec/UML

Os PDFs dos livros são material de referência do professor e **não** são publicados no deploy nem distribuídos aos alunos — cite-se sempre autor/capítulo/seção.

Ligação com as outras unidades

- **Vem da Unidade 2:** as Histórias de Usuário viram casos de uso; os cenários Gherkin viram diagramas de sequência.
- **Vem da Unidade 3:** o Site Map e os wireframes são conferidos contra o modelo de classes — é a verificação cruzada da aula 37. O IFML pressupõe o diagrama de classes como modelo estrutural.
- **Vai para a Unidade 5:** as regras que o diagrama de classes não consegue expressar são escritas em **OCL**.

Material legado reaproveitado

Origem em `../../legada/05-uml/`: `ES-07-Diagrama de Classe.pdf`, `ESw 08 - Diagrama Sequencia.pdf`, `aula9 - Diagrama de Atividades.pdf`, `apostila2uml.pdf`, `Diagrama de classe.docx` e os exercícios. O conteúdo foi reescrito, não copiado — as fontes citadas acima foram conferidas diretamente nos PDFs do acervo.

Manutenção (para quem edita a unidade)

- Os SVGs são inseridos **inline** pelo leitor, na mesma página. Depois de criar ou editar um SVG, rode `python3 ../isolar_svgs.py --aplicar`: ele torna únicos, por arquivo, os `id` e as classes CSS (sem isso, setas perdem a ponta e figuras herdam o estilo de outra). Não muda o desenho.
- `_staging/` guarda os fragmentos e relatórios do enriquecimento com o Vez (não vai ao deploy). `_staging/integrar.py` os integrou em `01-material-de-leitura.md` e `02-slides.md`; **não rode de novo** (duplicaria). Os originais anteriores estão em `_staging/backup/`.
- `python3 ../verificar_paridade.py` confere esta unidade contra o HTML antigo; os arquivos enriquecidos estão em `../paridade-evoluida.txt` (todas as palavras do HTML continuam obrigatórias).